

```

-- =====

-- DATABASE: ta_road_transport_agency

-- =====

CREATE DATABASE IF NOT EXISTS ta_road_transport_agency;
USE ta_road_transport_agency;

-- =====

-- TABLES CREATION (bez PRIMARY i FOREIGN KEY ograničenja)

-- =====

CREATE TABLE IF NOT EXISTS ta_drivers (
    driver_id INT,
    name VARCHAR(50),
    surname VARCHAR(50),
    license_number VARCHAR(20),
    hire_date DATE,
    experience_years INT,
    status ENUM('active', 'on_leave', 'inactive') DEFAULT 'active'
);

CREATE TABLE IF NOT EXISTS ta_vehicles (
    vehicle_id INT,
    plate_number VARCHAR(15),
    model VARCHAR(50),
    year YEAR,
    capacity_tons DECIMAL(5,1),
    fuel_type ENUM('Diesel', 'LPG', 'Petrol'),

```

```
status ENUM('available', 'in_service', 'maintenance') DEFAULT 'available'
);
```

```
CREATE TABLE IF NOT EXISTS ta_clients (
    client_id INT,
    company_name VARCHAR(100),
    contact_person VARCHAR(100),
    phone VARCHAR(30),
    email VARCHAR(100),
    country VARCHAR(100),
    city VARCHAR(100)
);
```

```
CREATE TABLE IF NOT EXISTS ta_shipments (
    shipment_id INT,
    client_id INT,
    origin VARCHAR(100),
    destination VARCHAR(100),
    distance_km DECIMAL(8,1),
    weight_tons DECIMAL(6,2),
    price_eur DECIMAL(10,2),
    date_requested DATE
);
```

```
CREATE TABLE IF NOT EXISTS ta_routes (
    route_id INT,
    shipment_id INT,
    driver_id INT,
```

```
vehicle_id INT,  
start_date DATE,  
end_date DATE,  
status ENUM('completed', 'in_progress', 'delayed') DEFAULT 'in_progress'  
);
```

```
CREATE TABLE IF NOT EXISTS ta_expenses (  
    expense_id INT,  
    route_id INT,  
    date DATE,  
    type ENUM('Fuel', 'Toll', 'Maintenance', 'Fine', 'Other'),  
    amount_eur DECIMAL(10,2),  
    description VARCHAR(255)  
);
```

```
CREATE TABLE IF NOT EXISTS ta_maintenance (  
    maintenance_id INT,  
    vehicle_id INT,  
    date DATE,  
    service_type ENUM('Oil Change', 'Tire Replacement', 'Engine Repair', 'Brake Service',  
'Other'),  
    cost_eur DECIMAL(10,2),  
    description VARCHAR(255)  
);
```

```
CREATE TABLE IF NOT EXISTS ta_fuel_logs (  
    fuel_log_id INT,  
    vehicle_id INT,
```

```
date DATE,  
liters DECIMAL(8,1),  
price_per_liter_eur DECIMAL(5,2),  
total_cost_eur DECIMAL(10,2),  
location VARCHAR(100)  
);
```

```
-- =====
```

```
-- POTOM IMPORT WIZARD PA DODAVANJE KLJUČEVA
```

```
-- =====
```

```
-- =====
```

```
-- DODAVANJE PRIMARY KEY I FOREIGN KEY OGRANIČENJA
```

```
-- =====
```

```
USE ta_road_transport_agency;
```

```
-- =====
```

```
-- PRIMARY KEYS
```

```
-- =====
```

```
ALTER TABLE ta_drivers
```

```
ADD PRIMARY KEY (driver_id);
```

```
ALTER TABLE ta_vehicles
```

```
ADD PRIMARY KEY (vehicle_id);
```

```
ALTER TABLE ta_clients
```

```
ADD PRIMARY KEY (client_id);
```

```
ALTER TABLE ta_shipments  
    ADD PRIMARY KEY (shipment_id);
```

```
ALTER TABLE ta_routes  
    ADD PRIMARY KEY (route_id);
```

```
ALTER TABLE ta_expenses  
    ADD PRIMARY KEY (expense_id);
```

```
ALTER TABLE ta_maintenance  
    ADD PRIMARY KEY (maintenance_id);
```

```
ALTER TABLE ta_fuel_logs  
    ADD PRIMARY KEY (fuel_log_id);
```

```
-- =====
```

```
-- FOREIGN KEYS
```

```
-- =====
```

```
ALTER TABLE ta_shipments  
    ADD CONSTRAINT fk_shipments_client  
    FOREIGN KEY (client_id)  
    REFERENCES ta_clients(client_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE;
```

```
ALTER TABLE ta_routes  
    ADD CONSTRAINT fk_routes_shipment  
    FOREIGN KEY (shipment_id)
```

```
REFERENCES ta_shipments(shipment_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

```
ALTER TABLE ta_routes
ADD CONSTRAINT fk_routes_driver
FOREIGN KEY (driver_id)
REFERENCES ta_drivers(driver_id)
ON DELETE SET NULL
ON UPDATE CASCADE;
```

```
ALTER TABLE ta_routes
ADD CONSTRAINT fk_routes_vehicle
FOREIGN KEY (vehicle_id)
REFERENCES ta_vehicles(vehicle_id)
ON DELETE SET NULL
ON UPDATE CASCADE;
```

```
ALTER TABLE ta_expenses
ADD CONSTRAINT fk_expenses_route
FOREIGN KEY (route_id)
REFERENCES ta_routes(route_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

```
ALTER TABLE ta_maintenance
ADD CONSTRAINT fk_maintenance_vehicle
FOREIGN KEY (vehicle_id)
```

```
REFERENCES ta_vehicles(vehicle_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

```
ALTER TABLE ta_fuel_logs
ADD CONSTRAINT fk_fuel_logs_vehicle
FOREIGN KEY (vehicle_id)
REFERENCES ta_vehicles(vehicle_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

```
-- =====
-- ROAD TRANSPORT AGENCY – 50 SQL QUESTIONS (ta_prefiks)
-- Autor: ChatGPT (GPT-5)
-- Opis: Komentarisana pitanja za vežbu MySQL upita
-- =====
```

```
-- =====
-- A. OSNOVNI SELECT UPITI
-- =====
```

```
-- 1. Prikaži sve vozače iz tabele ta_drivers.
```

```
SELECT
    CONCAT(d.name," ",d.surname) as Full_Name
FROM
    ta_drivers as d;
```

Result Grid				Filter
	Full_Name			
▶	Danielle Johnson			
	John Taylor			
	Erica McClain			
	Brittany Johnson			
	Jeffery Wagner			
	Anna Baldwin			
	Amy Robinson			
	Joshua Booth			
	Linda Wolfe			

Result 3 

-- 2. Prikaži ime, prezime i status svih vozača koji su trenutno aktivni.

```

SELECT
    CONCAT(d.name," ",d.surname) as Full_Name,
    d.status
FROM
    ta_drivers as d
WHERE
    d.status = "active";

```

Result Grid			 Filter Rows:
	Full_Name	status	
▶	Danielle Johnson	active	
	John Taylor	active	
	Erica McClain	active	
	Jeffery Wagner	active	
	Amy Robinson	active	
	Joshua Booth	active	
	Joshua Lewis	active	
	Matthew Davis	active	
	Andrea Calderon	active	

Result 4 ✕

Output

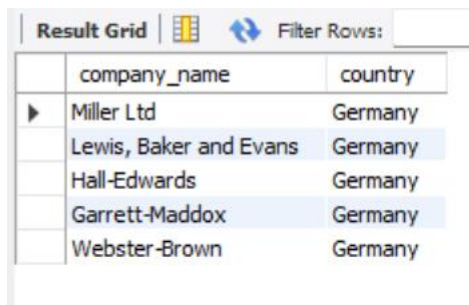
-- 3. Izlistaj sve klijente koji se nalaze u Nemačkoj (country = 'Germany').

```

SELECT

```

```
        c.company_name,  
        c.country  
FROM  
        ta_clients as c  
WHERE  
        c.country = 'Germany';
```



The screenshot shows a 'Result Grid' window with a toolbar containing icons for grid view, refresh, and a 'Filter Rows' input field. The grid displays two columns: 'company_name' and 'country'. There are five rows of data, all with 'Germany' in the 'country' column. The second, fourth, and fifth rows are highlighted in blue.

	company_name	country
▶	Miller Ltd	Germany
	Lewis, Baker and Evans	Germany
	Hall-Edwards	Germany
	Garrett-Maddox	Germany
	Webster-Brown	Germany

-- 4. Pronađi sva vozila proizvedena posle 2020. godine.

```
SELECT  
        v.vehicle_id,  
        v.model,  
        v.year  
FROM  
        ta_vehicles as v  
WHERE  
        v.year > 2020;
```

Result Grid			
Filter Rows:			
	vehide_id	model	year
▶	1	DAF XF	2021
	3	DAF XF	2024
	5	Scania R.500	2021
	6	DAF XF	2023
	7	Volvo FH16	2023
	9	Scania R.500	2023
	13	Volvo FH16	2022
	14	DAF XF	2023
	16	Mercedes Actros	2022

ta_vehicles 6 x

-- 5. Prikaži sve pošiljke teže od 10 tona.

SELECT

sh.shipment_id,

CONCAT(sh.origin,"-",sh.destination) as Rute,

sh.weight_tons,

sh.date_requested

FROM

ta_shipments as sh

WHERE

sh.weight_tons > 10

ORDER BY

sh.date_requested;

Result Grid				
Filter Rows:				
Export: Wrap Cell Content:				
	shipment_id	Rute	weight_tons	date_requested
▶	10345	New Vanessa-North Richard	16.55	2023-01-01
	634	South Tammy-Fosterport	21.79	2023-01-01
	826	South Tina-West Douglaschester	22.10	2023-01-01
	1114	Port Kelly-Floresport	19.34	2023-01-01
	3583	Gainesland-Port William	16.21	2023-01-01
	3995	Hectorport-South Ernestburgh	16.38	2023-01-01
	4321	Kennethville-North Vanessa	24.04	2023-01-01
	4519	Sarahmouth-New Teresa	11.16	2023-01-01
	4900	Chandlerton-Englishfurt	11.87	2023-01-01

Result 7 x

Output

-- 6. Prikaži pošiljke sa cenom većom od 5.000 EUR.

```
SELECT
    sh.shipment_id,
    CONCAT(sh.origin,"-",sh.destination) as Rute,
    sh.price_eur,
    sh.date_requested
FROM
    ta_shipments as sh
WHERE
    sh.price_eur > 5000
ORDER BY
    sh.date_requested;
```

shipment_id	Rute	price_eur	date_requested
3514	West Allisonville-New Jennifer	6326.63	2023-01-02
11765	North Melissa-North Brittanyfurt	6086.00	2023-01-02
4312	Heatherbury-South Timothyand	5320.66	2023-01-03
5067	Moniquechester-Nicholasfort	5246.30	2023-01-03
12293	South Rachelton-New Steven	6236.57	2023-01-04
8043	Jeffreyfort-Ashleyborough	6277.56	2023-01-08
12980	Wangside-Williamville	6292.60	2023-01-12
2901	Jacobport-Scottborough	5736.90	2023-01-17
3766	West Bianca-West Nicole	5895.18	2023-01-17

-- 7. Izlistaj sve rute koje su trenutno "in_progress".

```
SELECT
    sh.shipment_id,
    CONCAT(sh.origin,"-",sh.destination) as Rute,
```

```

sh.price_eur,
sh.date_requested,
r.`status`
FROM
    ta_shipments as sh
LEFT JOIN
    ta_routes as r ON sh.shipment_id = r.shipment_id
WHERE
    r.status = "in_progress"
ORDER BY
    sh.date_requested;

```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
	shipment_id	Route	price_eur	date_requested	status
▶	535	Wilsonstad-East Jennifer	226.51	2023-01-01	in_progress
	634	South Tammy-Fosterport	530.64	2023-01-01	in_progress
	3583	Gainesland-Port William	4146.81	2023-01-01	in_progress
	3625	East Todd-New Olivia	66.39	2023-01-01	in_progress
	4321	Kennethville-North Vanessa	267.50	2023-01-01	in_progress
	5238	East Jessemouth-Jonesland	1196.56	2023-01-01	in_progress
	5274	New Robert-Grantberg	1646.17	2023-01-01	in_progress
	4542	West Heatherside-East Kimberlyfurt	1155.33	2023-01-02	in_progress
	6086	South Zacharv-Schwartzfurt	286.03	2023-01-02	in prooress

-- 8. Prikaži sve troškove koji su tipa "Fuel" for models Skania R500.

```

SELECT
    exp.*,
    v.vehicle_id,
    v.model
FROM
    ta_expenses AS exp

```

LEFT JOIN

ta_routes AS r ON exp.route_id = r.route_id

LEFT JOIN

ta_vehicles AS v ON r.vehicle_id = v.vehicle_id

WHERE

exp.type = 'Fuel'

AND v.model = 'Scania R500';

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	expense_id	route_id	date	type	amount_eur	description	vehide_id	model
▶	5223	300	2024-03-06	Fuel	366.08	Run candidate upon system class.	2	Scania R500
	5817	544	2024-11-03	Fuel	148.53	Director time mention.	2	Scania R500
	6283	725	2024-12-31	Fuel	379.36	Scene painting set charge ball.	2	Scania R500
	58	1040	2024-12-20	Fuel	49.32	Young Democrat gun purpose.	2	Scania R500
	10638	1083	2024-06-23	Fuel	54.68	Eight floor the able.	2	Scania R500
	8163	1276	2024-01-04	Fuel	187.95	Back interest five century win.	2	Scania R500
	8164	1276	2024-01-05	Fuel	353.99	Song drive information admit.	2	Scania R500
	8165	1276	2024-01-03	Fuel	220.43	Business hair plan movie fear.	2	Scania R500
	8517	1435	2023-09-09	Fuel	227.45	Sure attorney employee strong.	2	Scania R500

Result 10

-- 9. Prikaži sve servise gde je cena održavanja bila preko 1.000 EUR, a u pitanju su vozila na servisu.

SELECT

m.*

FROM ta_maintenance as m

LEFT JOIN ta_vehicles as v ON m.vehicle_id = v.vehicle_id

WHERE

m.cost_eur > 1000 AND v.status = 'in_service';

Result Grid						
Filter Rows:			Export:		Wrap Cell Content:	
	maintenance_id	vehicle_id	date	service_type	cost_eur	description
▶	16	5	2024-04-03	Oil Change	1738.73	Nice fine just finish attention strong inside.
	17	5	2023-06-01	Tire Replacement	1772.56	Smile religious course voice campaign entire.
	67	19	2023-06-01	Oil Change	1591.06	Arm generation business prepare full industry.
	82	24	2023-07-18	Tire Replacement	1058.33	Audience approach cover design reason campai...
	83	24	2024-12-12	Tire Replacement	1995.54	Value subject else.
	84	25	2023-09-24	Brake Service	1665.39	Positive shake official future election.
	85	25	2023-10-01	Brake Service	1753.11	Body huge direction call arm challenge.
	87	25	2023-01-22	Brake Service	1984.01	Feeling condition scene station plant.
	92	28	2023-04-04	Engine Repair	1317.09	Interesting world else Democrat claim rock.

-- 10. Pronađi sve točenja goriva gde je litraža veća od 500 litara.

SELECT

*

FROM

ta_fuel_logs as fl

WHERE

fl.liters > 500;

Result Grid							
Filter Rows:			Edit:		Export/Import:		Wrap Cell Content:
	fuel_log_id	vehicle_id	date	liters	price_per_liter_eur	total_cost_eur	location
▶	5	1	2023-12-28	597.0	1.75	1044.75	East Robertmouth
	7	1	2024-02-20	519.9	1.87	972.21	New Christopherton
	8	1	2023-06-09	505.5	1.71	864.40	Lake Michael
	10	1	2024-04-28	541.3	1.87	1012.23	West Brandon
	17	1	2023-03-30	593.0	1.77	1049.61	North Christopherborough
	25	1	2023-10-04	556.8	1.56	868.61	Crawfordshire
	26	1	2023-03-09	590.7	1.59	939.21	New Heatherhaven
	32	1	2023-05-31	535.0	1.50	802.50	Christopherstad
	43	1	2024-05-12	558.9	1.53	855.12	North Nancvtown

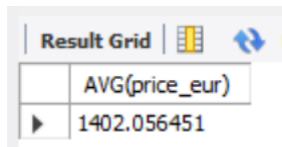
-- =====

-- B. AGREGATNE FUNKCIJE I GRUPE

-- =====

-- 11. Pronađi prosečnu cenu pošiljke (AVG(price_eur)).

```
SELECT  
  
    AVG(price_eur)  
  
FROM  
  
    ta_shipments as s;
```

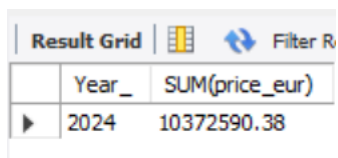


A screenshot of a database interface showing a 'Result Grid'. The grid has two columns: 'AVG(price_eur)' and a value '1402.056451'. There are icons for 'Result Grid', a grid icon, and a refresh icon.

	AVG(price_eur)
▶	1402.056451

-- 12. Pronađi ukupnu vrednost svih pošiljki u 2024. godini.

```
SELECT  
  
    YEAR(s.date_requested) as Year_,  
  
    SUM(price_eur)  
  
FROM  
  
    ta_shipments as s  
  
GROUP BY  
  
    Year_  
  
HAVING  
  
    Year_ = 2024;
```

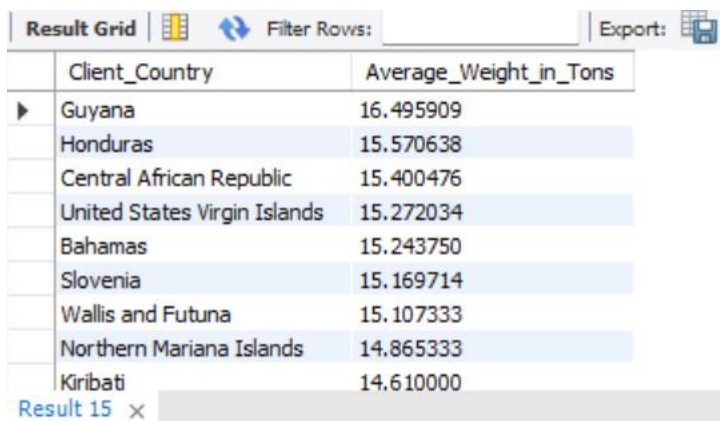


A screenshot of a database interface showing a 'Result Grid'. The grid has three columns: 'Year_', 'SUM(price_eur)', and a value '10372590.38'. There are icons for 'Result Grid', a grid icon, a refresh icon, and a 'Filter R' button.

	Year_	SUM(price_eur)
▶	2024	10372590.38

-- 13. Izračunaj prosečnu težinu pošiljke po državi klijenta.

```
SELECT
    c.country as Client_Country,
    AVG(s.weight_tons) as Average_Weight_in_Tons
FROM
    ta_shipments as s
LEFT JOIN
    ta_clients as c
ON s.client_id = c.client_id
GROUP BY
    Client_Country
ORDER BY
    Average_Weight_in_Tons DESC;
```



Result Grid | Filter Rows: | Export:

	Client_Country	Average_Weight_in_Tons
▶	Guyana	16.495909
	Honduras	15.570638
	Central African Republic	15.400476
	United States Virgin Islands	15.272034
	Bahamas	15.243750
	Slovenia	15.169714
	Wallis and Futuna	15.107333
	Northern Mariana Islands	14.865333
	Kiribati	14.610000

Result 15 x

-- 14. Pronađi najdužu ,najkraću i prosečnu udaljenost (MAX i MIN i AVG distance_km) po klijentu.

```
SELECT
    c.company_name,
    MAX(s.distance_km) as Max_Distance_km,
```

```

    AVG(s.distance_km) as Average_Distance_km,
    MIN(s.distance_km) as Min_Distance_km
FROM
    ta_shipments as s
LEFT JOIN
    ta_clients as c
ON
    s.client_id = c.client_id
GROUP BY
    c.company_name
ORDER BY
    Average_Distance_km DESC;

```

Result Grid				
		Filter Rows:	Export:	Wrap Cell Content: IA
	company_name	Max_Distance_km	Average_Distance_km	Min_Distance_km
▶	Moore-Harris	2447.3	1635.72778	293.3
	Moore-Collins	2421.4	1591.08000	501.7
	Diaz-Vega	2491.2	1590.07143	269.5
	Harris Inc	2439.8	1542.99615	129.7
	Dunn LLC	2444.7	1526.30968	130.6
	Mora-White	2495.6	1516.44194	275.5
	Thompson, Lang and Oneill	2485.9	1516.22308	218.7
	Nunez LLC	2498.6	1514.95429	225.0
	Norman PLC	2473.2	1514.81200	207.0

-- 15. Izračunaj koliko ukupno ima vozača po statusu.

```

SELECT
    d.status,
    COUNT(d.driver_id) as Total_Drivers
FROM
    ta_drivers as d

```

GROUP BY

d.status

ORDER BY

Total_Drivers;

-- 16. Pronađi ukupno sipano gorivo po modelu vozila i mesecu sipanja.

SELECT

v.model,

MONTH(fl.date) AS MonthOfFuelLogs,

SUM(fl.liters) AS Total_Fuel_in_Liters

FROM

ta_fuel_logs AS fl

LEFT JOIN

ta_vehicles AS v ON fl.vehicle_id = v.vehicle_id

GROUP BY

v.model,

MONTH(fl.date)

ORDER BY

v.model,

MONTH(fl.date) ASC;

Result Grid				Filter Rows:	Export:
	model	MonthOfFuelLogs	Total_Fuel_in_Liters		
▶	DAF XF	1	34160.5		
	DAF XF	2	32590.6		
	DAF XF	3	37072.4		
	DAF XF	4	34651.7		
	DAF XF	5	28166.6		
	DAF XF	6	34564.8		
	DAF XF	7	32133.2		
	DAF XF	8	30714.8		
	DAF XF	9	26545.1		
Result 17				Result 18 x	

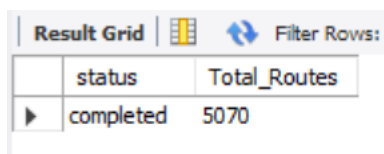
-- 17. Najčešća godina proizvodnje vozila po modelu

```
SELECT
    v1.model,
    v1.year AS most_common_year,
    COUNT(*) AS count_per_year
FROM
    ta_vehicles v1
GROUP BY
    v1.model, v1.year
HAVING
    COUNT(*) = (
        SELECT
            MAX(COUNT(*))
        FROM
            ta_vehicles v2
        WHERE
            v2.model = v1.model
        GROUP BY
            v2.year
    )
ORDER BY
    v1.model;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Co
model	most_common_year	count_per_year	
DAF XF	2021	2	
DAF XF	2023	2	
Iveco S-Way	2016	1	
Iveco S-Way	2020	1	
Iveco S-Way	2019	1	
Iveco S-Way	2018	1	
Iveco S-Way	2023	1	
MAN TGX	2018	1	
MAN TGX	2015	1	

-- 17. Koliko je ukupno ruta završeno (status = 'completed')?

```
SELECT
    r.status,
    COUNT(route_id) as Total_Routes
FROM
    ta_routes as r
GROUP BY
    r.status
HAVING
    r.status = 'completed';
```



The screenshot shows a database query result grid with two columns: 'status' and 'Total_Routes'. There is one row of data where 'status' is 'completed' and 'Total_Routes' is 5070. The grid has a header row and a data row. Above the grid, there are tabs for 'Result Grid' and 'Filter Rows:'. The 'Result Grid' tab is active.

	status	Total_Routes
▶	completed	5070

-- 18. Koliko je ukupno potrošeno na gorivo (SUM(amount_eur) gde type='Fuel')?

```
SELECT
    e.type,
    SUM(e.amount_eur) as TotalCost
FROM
    ta_expenses as e
GROUP BY
    e.type
HAVING
    e.type = 'Fuel';
```

Result Grid			Filter
	type	TotalCost	
▶	Fuel	798484.72	

-- 19. Koliko ukupno ima točenja goriva po vozilu?

```

SELECT
    v.plate_number,
    COUNT(fl.fuel_log_id) as TotalFuelLogs
FROM
    ta_fuel_logs as fl
LEFT JOIN
    ta_vehicles as v
ON
    fl.vehicle_id = v.vehicle_id
GROUP BY
    v.plate_number
ORDER BY
    COUNT(fl.fuel_log_id) DESC;

```

Result Grid			Filter Rows:
	plate_number	TotalFuelLogs	
▶	XP-742-C	199	
	AS-353-R	199	
	UU-620-D	191	
	YR-710-Y	176	
	GR-354-E	174	
	IQ-880-T	168	
	LZ-371-J	161	
	XS-287-S	158	
	RO-787-E	157	
Result 23			×

-- 20. Koliki je prosečan trošak po ruti (AVG(amount_eur) po route_id)?

```
SELECT
    r.route_id,
    CONCAT(sh.origin,"-",sh.destination) as Rute,
    AVG(e.amount_eur) as AverageCostPerRute
FROM
    ta_shipments as sh
LEFT JOIN
    ta_routes as r ON sh.shipment_id = r.shipment_id
LEFT JOIN
    ta_expenses as e ON r.route_id = e.route_id
GROUP BY
    r.route_id,
    Rute
HAVING
    AVG(e.amount_eur) IS NOT NULL
ORDER BY
    AVG(e.amount_eur) DESC;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Co
route_id	Rute	AverageCostPerRute	
2547	Port Robert-South Kristiville	494.680000	
8281	Brewerborough-New Cynthia	494.550000	
1428	Warrentown-New Andre	494.410000	
6373	Mirandaborough-North Jeremy	494.300000	
1784	Beckerside-Thompsonberg	493.970000	
11403	Katieland-Port Joseph	493.880000	
11779	Rochamouth-Stantontown	493.740000	
14549	South John-Colemanside	493.210000	
1737	Estradahaven-Jordanshire	492.930000	

Result 24 x

-- =====

-- C. JOIN UPITI

-- =====

-- 21. Prikaži listu ruta sa imenima vozača i registarskim brojem vozila.

SELECT

 r.route_id,

 r.start_date,

 CONCAT(sh.origin,"-",sh.destination) as Rute,

 CONCAT(d.name," ",d.surname) as FullDriverName,

 v.plate_number,

 v.model

FROM

 ta_shipments as sh

LEFT JOIN

 ta_routes as r ON sh.shipment_id = r.shipment_id

LEFT JOIN

 ta_drivers as d ON r.driver_id = d.driver_id

LEFT JOIN

 ta_vehicles as v ON r.vehicle_id = v.vehicle_id

ORDER BY

 r.start_date ASC;

route_id	start_date	Rute	FullDriverName	plate_number	model
10298	2023-01-01	Sharonmouth-Lake Terriland	Matthew Carter	CB-903-K	DAF XF
10345	2023-01-01	New Vanessa-North Richard	Linda Wolfe	UB-785-H	DAF XF
3583	2023-01-01	Gainesland-Port William	Veronica Bowman	KZ-708-D	Volvo FH16
3625	2023-01-01	East Todd-New Olivia	Linda Wolfe	KZ-708-D	Volvo FH16
3995	2023-01-01	Hectorport-South Ernestburgh	Randall Rocha	EE-838-V	Volvo FH16
4321	2023-01-01	Kennethville-North Vanessa	Nicole Ward	NN-294-O	Mercedes Actros
4519	2023-01-01	Sarahmouth-New Teresa	Jeffrey Bright	HG-309-G	Iveco S-Way
4900	2023-01-01	Chandleron-Englishfurt	Erin Carlson	EE-838-V	Volvo FH16
5139	2023-01-01	Sabrinaborough-Diazside	Veronica Bowman	RO-787-E	Iveco S-Way

-- 22. Prikaži sve pošiljke zajedno u jednoj celiji sa imenom kompanije klijenta koji ih je naručio.

SELECT

c.company_name,

GROUP_CONCAT(DISTINCT s.shipment_id ORDER BY s.shipment_id ASC) as
AllShipments

FROM

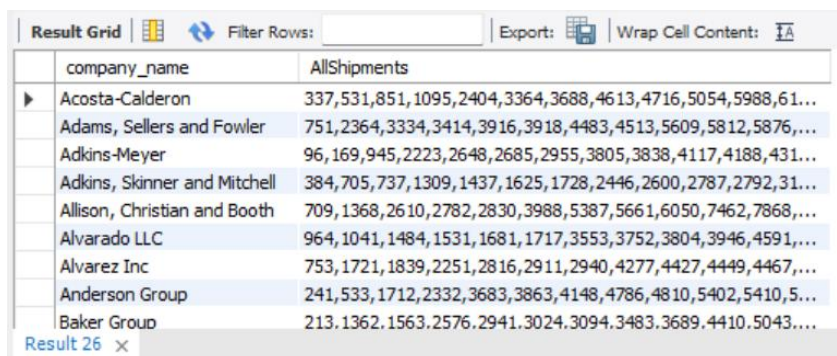
ta_shipments as s

LEFT JOIN

ta_clients as c ON s.client_id = c.client_id

GROUP BY

c.company_name;



The screenshot shows a 'Result Grid' with two columns: 'company_name' and 'AllShipments'. The 'AllShipments' column contains a long string of shipment IDs for each company, separated by commas. The companies listed are Acosta-Calderon, Adams, Sellers and Fowler, Adkins-Meyer, Adkins, Skinner and Mitchell, Allison, Christian and Booth, Alvarado LLC, Alvarez Inc, Anderson Group, and Baker Group. The grid also includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox.

company_name	AllShipments
Acosta-Calderon	337,531,851,1095,2404,3364,3688,4613,4716,5054,5988,61...
Adams, Sellers and Fowler	751,2364,3334,3414,3916,3918,4483,4513,5609,5812,5876,...
Adkins-Meyer	96,169,945,2223,2648,2685,2955,3805,3838,4117,4188,431...
Adkins, Skinner and Mitchell	384,705,737,1309,1437,1625,1728,2446,2600,2787,2792,31...
Allison, Christian and Booth	709,1368,2610,2782,2830,3988,5387,5661,6050,7462,7868,...
Alvarado LLC	964,1041,1484,1531,1681,1717,3553,3752,3804,3946,4591,...
Alvarez Inc	753,1721,1839,2251,2816,2911,2940,4277,4427,4449,4467,...
Anderson Group	241,533,1712,2332,3683,3863,4148,4786,4810,5402,5410,5...
Baker Group	213.1362.1563.2576.2941.3024.3094.3483.3689.4410.5043....

-- 23. Prikazi sve troškove (expenses) po gradu polaska pošiljke.

SELECT

sh.origin,

SUM(e.amount_eur) as TotalExpenses

FROM

ta_shipments as sh

LEFT JOIN

```

        ta_routes as r ON sh.shipment_id = r.shipment_id
LEFT JOIN
        ta_expenses as e ON r.route_id = e.route_id
GROUP BY
        sh.origin
HAVING
        SUM(e.amount_eur) IS NOT NULL
ORDER BY
        SUM(e.amount_eur) DESC;

```

Result Grid		
	origin	TotalExpenses
▶	Port Michael	5223.24
	Robertmouth	4359.06
	Lake Jessica	4263.85
	New Michael	4124.71
	South Christopher	4029.40
	Michaelmouth	3976.70
	East Jennifer	3956.38
	South Stephanie	3892.16
	Sarahport	3684.26

-- 24. Prikaži sve servise (maintenance) zajedno sa modelom vozila na kojem su rađeni.

```

SELECT
        v.model,
        m.service_type,
        SUM(m.cost_eur) as TotalServiceCost
FROM
        ta_maintenance as m
LEFT JOIN
        ta_vehicles as v ON m.vehicle_id = v.vehicle_id
GROUP BY

```

v.model,
m.service_type;

Result Grid	Filter Rows:	Export:	Wr
	model	service_type	TotalServiceCost
▶	DAF XF	Brake Service	8330.44
	DAF XF	Tire Replacement	9244.55
	Scania R500	Tire Replacement	8750.65
	Scania R500	Engine Repair	2723.66
	Scania R500	Oil Change	8426.05
	Scania R500	Brake Service	2275.56
	DAF XF	Oil Change	7590.43
	DAF XF	Engine Repair	5460.39
	Volvo FH16	Oil Change	7116.44

-- 25. Prikaži sve rute, zajedno sa imenom vozača i ukupnim troškovima putarine za tu rutu.

```
SELECT
    CONCAT(sh.origin, ' - ', sh.destination) AS Rute,
    CONCAT(d.name, ' ', d.surname) AS FullDriverName,
    SUM(e.amount_eur) AS TotalExpenses
FROM
    ta_shipments AS sh
LEFT JOIN
    ta_routes AS r ON sh.shipment_id = r.shipment_id
LEFT JOIN
    ta_drivers AS d ON r.driver_id = d.driver_id
LEFT JOIN
    ta_vehicles AS v ON r.vehicle_id = v.vehicle_id
LEFT JOIN
    ta_expenses AS e ON r.route_id = e.route_id
WHERE
```

e.type = 'Toll'

GROUP BY

Rute, FullDriverName

ORDER BY

TotalExpenses DESC;

Result Grid	Filter Rows:	Export:	Wrap Cell Content
	Rute	FullDriverName	TotalExpenses
▶	Port Bradley - East Michael	Chelsea Jackson	1266.90
	Courtneyberg - Walkerturt	Linda Morales	1195.60
	Richport - New Ianshire	Daniel Burton	1155.53
	Janeburgh - Smithberg	Mitchell Clark	1119.93
	Wendybury - West Brent	Alyssa Smith	1118.91
	Angelaton - Warrenstad	Anna Baker	1112.28
	Barbaraborough - Kaylatown	Jennifer Stanton	1109.29
	Elizabethview - Port Dawnbury	Douglas Taylor	1057.55
	Tammvbero - Erintown	Jacqueline Sutton	1007.77

Result 29 x

-- 26. Prikaži sve pošiljke koje su prevezene vozilom marke "Volvo FH16".

SELECT

v.model,

r.route_id,

sh.shipment_id,

CONCAT(sh.origin, ' - ', sh.destination) AS Rute

FROM

ta_routes as r

LEFT JOIN

ta_vehicles as v ON r.vehicle_id = v.vehicle_id

LEFT JOIN

ta_shipments AS sh ON r.shipment_id = sh.shipment_id

WHERE v.model = 'Volvo FH16';

Result Grid				
		Filter Rows:	Export:	Wrap Cell Content:
	model	route_id	shipment_id	Rute
▶	Volvo FH16	4	4	Brianmouth - Eatonchester
	Volvo FH16	9	9	Port Zacharyville - Blakechester
	Volvo FH16	22	22	Robertfurt - Knightport
	Volvo FH16	70	70	North Shawntown - Andrestad
	Volvo FH16	84	84	West Erikside - North Matthew
	Volvo FH16	265	265	South Carolburgh - East Adriennemouth
	Volvo FH16	288	288	Lake Dennisland - East Catherine
	Volvo FH16	315	315	Elizabethland - Jamesview
	Volvo FH16	361	361	Castilloview - East Mariah

Result 30 x

-- 27. Prikaži sve vozače i broj ruta koje su odvezli.

SELECT

CONCAT(d.name," ",d.surname) as FullDriverName,

COUNT(r.route_id) as NumberOfRoutes

FROM

ta_routes as r

LEFT JOIN

ta_drivers as d ON r.driver_id = d.driver_id

GROUP BY

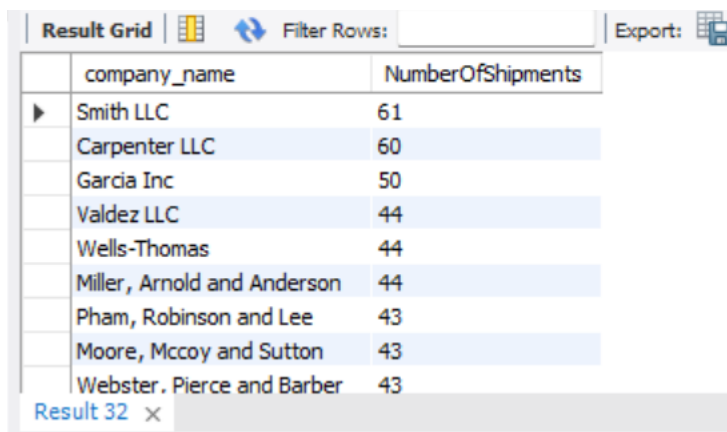
FullDriverName;

Result Grid		
		Filter Rows:
	FullDriverName	NumberOfRoutes
▶	Danielle Johnson	295
	John Taylor	303
	Erica McClain	301
	Brittany Johnson	310
	Jeffery Wagner	281
	Anna Baldwin	292
	Amy Robinson	300
	Joshua Booth	309
	Linda Wolfe	275

Result 31 x

-- 28. Prikaži sve klijente i broj pošiljki koje su naručili.

```
SELECT
    c.company_name,
    COUNT(s.shipment_id) as NumberOfShipments
FROM
    ta_shipments as s
LEFT JOIN
    ta_clients as c ON s.client_id = c.client_id
GROUP BY
    c.company_name
ORDER BY
    COUNT(s.shipment_id) DESC;
```



The screenshot shows a database query result grid with two columns: 'company_name' and 'NumberOfShipments'. The results are ordered by the number of shipments in descending order. The first row is 'Smith LLC' with 61 shipments, followed by 'Carpenter LLC' with 60, 'Garcia Inc' with 50, 'Valdez LLC' with 44, 'Wells-Thomas' with 44, 'Miller, Arnold and Anderson' with 44, 'Pham, Robinson and Lee' with 43, 'Moore, McCooy and Sutton' with 43, and 'Webster, Pierce and Barber' with 43. The interface includes a 'Result Grid' tab, a 'Filter Rows' field, and an 'Export' button.

company_name	NumberOfShipments
Smith LLC	61
Carpenter LLC	60
Garcia Inc	50
Valdez LLC	44
Wells-Thomas	44
Miller, Arnold and Anderson	44
Pham, Robinson and Lee	43
Moore, McCooy and Sutton	43
Webster, Pierce and Barber	43

-- 29. Prikaži sva vozila i ukupnu količinu goriva koje su natočili.

```
SELECT
    v.plate_number,
    SUM(fl.liters) as TotalFuelLitres,
    RANK() OVER (ORDER BY SUM(fl.liters) DESC) as RankOfTotalFuelLitres
FROM
```

```

        ta_fuel_logs as fl

LEFT JOIN

        ta_vehicles as v ON fl.vehicle_id = v.vehicle_id

GROUP BY

        v.plate_number

ORDER BY

        TotalFuelLitres DESC;

```

Result Grid			
		Filter Rows:	Export:
	plate_number	TotalFuelLitres	RankOfTotalFuelLitres
▶	XP-742-C	70033.4	1
	AS-353-R	68163.0	2
	UU-620-D	66758.7	3
	YR-710-Y	61793.4	4
	IQ-880-T	60552.3	5
	GR-354-E	59513.7	6
	LZ-371-J	56598.0	7
	RO-787-E	54854.5	8
	PA-771-D	52996.0	9

-- 30. Pronađi sve vozače koji trenutno imaju neku rutu u toku (status='in_progress').

```

SELECT

    DISTINCT d.driver_id,

    CONCAT(d.name, ' ', d.surname) AS FullDriverName

FROM

    ta_routes as r

LEFT JOIN

    ta_drivers as d ON r.driver_id = d.driver_id

WHERE

    r.status = 'in_progress';

```

Result Grid			Filter Rows:
	driver_id	FullDriverName	
▶	36	Trevor Campos	
	50	Alyssa Smith	
	47	Colleen Skinner	
	33	Anna Baker	
	7	Amy Robinson	
	22	Jacqueline Sutton	
	31	Randall Rocha	
	19	Jesse Garcia	
	1	Danielle Johnson	
Result 34			×

```
-- =====
```

```
-- D. FILTRIRANJE, USLOVI I PODUPITI
```

```
-- =====
```

```
-- 31. Prikaži sve pošiljke koje su skuplje od prosečne cene svih pošiljki.
```

```
WITH shiping_average_price AS (
    SELECT
        AVG(s.price_eur) AS AverageShipingPrice
    FROM
        ta_shipments AS s
)
SELECT
    s1.shipment_id,
    CONCAT(s1.origin,'-',s1.destination) as Rute,
    s1.price_eur,
    avg_price.AverageShipingPrice
FROM
    ta_shipments AS s1
CROSS JOIN
```

shipping_average_price AS avg_price

WHERE

s1.price_eur > avg_price.AverageShippingPrice

ORDER BY

s1.price_eur DESC;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch
shipment_id	Route	price_eur	AverageShippingPrice	
1281	Mikefurt-South Margaret	7188.56	1402.056451	
13860	Scottville-Jacobsonland	7121.51	1402.056451	
4948	Cummingsmouth-West Kimberly	7023.84	1402.056451	
3480	West Deborah-Port Russellfort	6754.88	1402.056451	
12333	Stephaniebury-South Amanda	6667.26	1402.056451	
12549	Harrismouth-Hannashire	6592.62	1402.056451	
10893	West Deannafurt-West Robert	6576.45	1402.056451	
1787	West Shawnhaven-Garymouth	6562.94	1402.056451	
14962	North Michaelburh-Barbaraview	6558.11	1402.056451	

Result 35 x

-- 32. Pronadi vozila koja nikada nisu imala evidentiran servis.

SELECT

v.plate_number,

SUM(m.cost_eur) as TotalMaintanceCost

FROM

ta_vehicles as v

LEFT JOIN

ta_maintenance as m ON v.vehicle_id = m.vehicle_id

GROUP BY

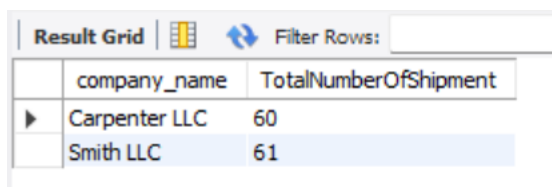
v.plate_number

HAVING

SUM(m.cost_eur) IS NULL;

-- 33. Pronađi klijente koji imaju više od 50 pošiljki.

```
SELECT
    c.company_name,
    COUNT(s.shipment_id) as TotalNumberOfShipment
FROM
    ta_clients as c
LEFT JOIN
    ta_shipments as s ON c.client_id = s.client_id
GROUP BY
    c.company_name
HAVING
    COUNT(s.shipment_id) > 50;
```



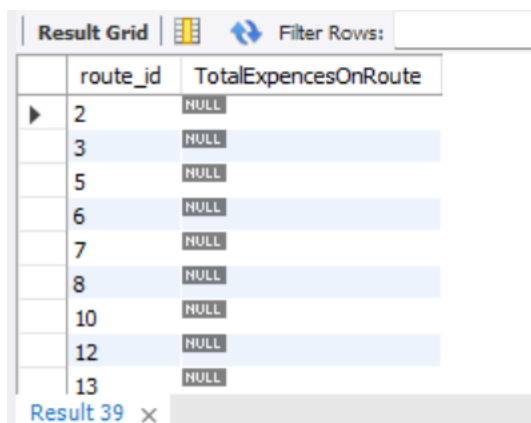
	company_name	TotalNumberOfShipment
▶	Carpenter LLC	60
	Smith LLC	61

-- 34. Prikaži vozače koji nisu imali nijednu završenu rutu.

```
SELECT
    CONCAT(d.name,' ',d.surname) AS FullDriverName
FROM
    ta_drivers as d
LEFT JOIN
    ta_routes as r ON d.driver_id = r.driver_id AND r.status = 'completed'
WHERE
    r.route_id IS NULL;
```

-- 35. Pronađi rute koje nisu imale evidentirane troškove.

```
SELECT
    r.route_id,
    SUM(e.amount_eur) as TotalExpencesOnRoute
FROM
    ta_routes as r
LEFT JOIN
    ta_expenses as e ON r.route_id = e.route_id
GROUP BY
    r.route_id
HAVING
    SUM(e.amount_eur) IS NULL;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains two columns: 'route_id' and 'TotalExpencesOnRoute'. There are 10 rows of data, all showing 'NULL' for the total expenses. The rows are numbered 2 through 13. The status bar at the bottom indicates 'Result 39'.

route_id	TotalExpencesOnRoute
2	NULL
3	NULL
5	NULL
6	NULL
7	NULL
8	NULL
10	NULL
12	NULL
13	NULL

-- 36. Prikaži vozila koja su koristila oba tipa goriva ("Diesel" i "LPG").

```
SELECT
    v.plate_number
FROM
```

```
ta_vehicles AS v

WHERE

    v.fuel_type IN ('Diesel', 'LPG')

GROUP BY

    v.plate_number

HAVING

    COUNT(DISTINCT v.fuel_type) = 2

ORDER BY

    v.plate_number ASC;
```

-- 37. Pronađi rutu sa najvećim ukupnim troškom.

```
SELECT

    r.route_id,

    CONCAT(sh.origin, ' - ', sh.destination) AS Rute,

    SUM(e.amount_eur) AS TotalExpenses

FROM

    ta_routes AS r

LEFT JOIN

    ta_expenses AS e ON r.route_id = e.route_id

LEFT JOIN

    ta_shipments AS sh ON r.shipment_id = sh.shipment_id

GROUP BY

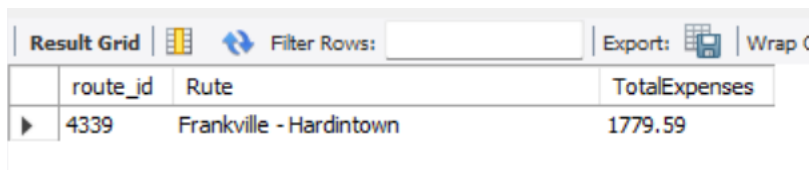
    r.route_id,

    CONCAT(sh.origin, ' - ', sh.destination)

ORDER BY

    TotalExpenses DESC
```

LIMIT 1;



The screenshot shows a database query result grid. At the top, there are tabs for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap'. Below the tabs is a table with three columns: 'route_id', 'Rute', and 'TotalExpenses'. The first row of data shows '4339' for route_id, 'Frankville - Hardintown' for Rute, and '1779.59' for TotalExpenses.

route_id	Rute	TotalExpenses
4339	Frankville - Hardintown	1779.59

-- 38. Pronađi vozilo sa najvećom ukupnom potrošnjom goriva (po liters).

SELECT

v.plate_number,

SUM(fl.liters) as TotalLitersLoad

FROM

ta_vehicles AS v

LEFT JOIN

ta_fuel_logs as fl ON v.vehicle_id = fl.vehicle_id

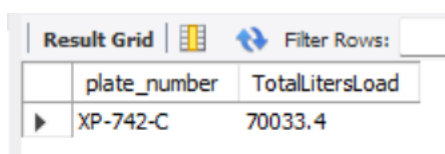
GROUP BY

v.plate_number

ORDER BY

SUM(fl.liters) DESC

LIMIT 1;



The screenshot shows a database query result grid. At the top, there are tabs for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap'. Below the tabs is a table with two columns: 'plate_number' and 'TotalLitersLoad'. The first row of data shows 'XP-742-C' for plate_number and '70033.4' for TotalLitersLoad.

plate_number	TotalLitersLoad
XP-742-C	70033.4

-- 39. Pronađi klijenta koji je ostvario najveći promet (SUM(price_eur)).

SELECT

c.company_name,

SUM(s.price_eur) as TotalRevenue

FROM

```

        ta_clients as c
LEFT JOIN
        ta_shipments as s ON c.client_id = s.client_id
GROUP BY
        c.company_name
ORDER BY
        SUM(s.price_eur) DESC
LIMIT 1;

```

Result Grid		Filter Rows:
	company_name	TotalRevenue
▶	Smith LLC	97103.32

-- 40. Prikaži top 5 najskupljih pošiljki sa imenom klijenta i destinacijom.

```

SELECT
        c.company_name,
        s.destination,
        s.price_eur
FROM
        ta_clients as c
LEFT JOIN
        ta_shipments as s ON c.client_id = s.client_id
ORDER BY
        s.price_eur DESC
LIMIT 5;

```

Result Grid		Filter Rows:	Export:
	company_name	destination	price_eur
▶	Morales-Calhoun	South Margaret	7188.56
	Becker PLC	Jacobsonland	7121.51
	Johnson, Schmitt and Miles	West Kimberly	7023.84
	Richardson, Good and Carroll	Port Russellfort	6754.88
	Jensen Inc	South Amanda	6667.26

-- =====

-- E. NAPREDNI UPITI I ANALIZE

-- =====

-- 41. Izračunaj prosečan trošak goriva po kilometru (ukupni troškovi goriva / ukupna kilometraža).

SELECT

SUM(e.amount_eur) AS TotalFuelCosts,

SUM(s.distance_km) AS TotalKM,

(SUM(e.amount_eur) / SUM(s.distance_km)) AS FuelCostsPerKm

FROM

ta_expenses AS e

LEFT JOIN

ta_routes AS r ON e.route_id = r.route_id

LEFT JOIN

ta_shipments AS s ON r.shipment_id = s.shipment_id

WHERE

e.type = 'Fuel';

Result Grid				Filter Rows:
	TotalFuelCosts	TotalKM	FuelCostsPerKm	
▶	798484.72	3893105.8	0.205102	

-- 42. Izračunaj prosečan prihod po ruti (SUM(price_eur)/COUNT(route_id)).

SELECT

SUM(s.price_eur) as TotalRevenue,

COUNT(r.route_id) as TotalNumberOfRoutes,

(SUM(s.price_eur) / COUNT(r.route_id)) as AverageRevenuePerRoute

FROM

ta_shipments as s

INNER JOIN

ta_routes as r ON s.shipment_id = r.shipment_id;

Result Grid	Filter Rows:	Export:	Wrap
	TotalRevenue	TotalNumberOfRoutes	AverageRevenuePerRoute
▶	21030846.77	15000	1402.056451

-- 43. Pronađi prosečnu težinu pošiljke po tipu vozila (JOIN ta_routes i ta_vehicles).

SELECT

v.model,

AVG(s.weight_tons) as AverageTonsLoad

FROM

ta_routes as r

LEFT JOIN

ta_vehicles as v ON r.vehicle_id = v.vehicle_id

LEFT JOIN

ta_shipments as s ON r.shipment_id = s.shipment_id

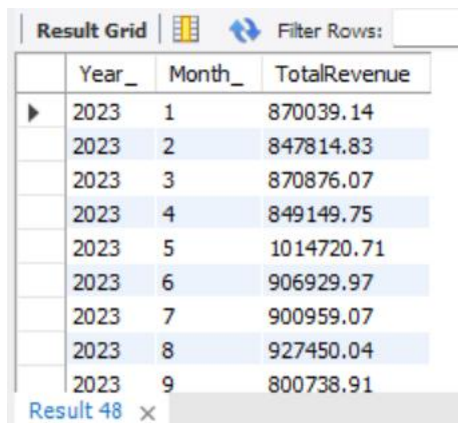
GROUP BY

v.model;

Result Grid				Filter Rows:	
	model	AverageTonsLoad			
▶	DAF XF	12.907862			
	Scania R500	13.077715			
	MAN TGX	13.140541			
	Volvo FH16	13.268197			
	Mercedes Actros	12.825630			
	Iveco S-Way	13.172380			

-- 44. Izračunaj mesečni prihod uključujući i potencijalan prihod agencije po godini i mesecu (GROUP BY YEAR, MONTH).

```
SELECT
    YEAR(date_requested) as Year_,
    MONTH(date_requested) as Month_,
    SUM(price_eur) as TotalRevenue
FROM
    ta_shipments as s
GROUP BY
    YEAR(date_requested),
    MONTH(date_requested)
ORDER BY
    YEAR(date_requested),
    MONTH(date_requested);
```



The screenshot shows a 'Result Grid' window with a table of monthly revenue for the year 2023. The table has three columns: 'Year_', 'Month_', and 'TotalRevenue'. The data is as follows:

Year_	Month_	TotalRevenue
2023	1	870039.14
2023	2	847814.83
2023	3	870876.07
2023	4	849149.75
2023	5	1014720.71
2023	6	906929.97
2023	7	900959.07
2023	8	927450.04
2023	9	800738.91

At the bottom of the window, it says 'Result 48' with a close button (x).

-- 45. Prikaži ukupne troškove održavanja po vozilu i prosečno vreme između servisa.

```
WITH ServiceIntervals AS (
    SELECT
```

```

    vehicle_id,

    date,

    LAG(date) OVER (PARTITION BY vehicle_id ORDER BY date) AS PreviousServiceDate

FROM

    ta_maintenance

)

SELECT

    m.vehicle_id,

    SUM(m.cost_eur) AS TotalMaintenanceCost,

    ROUND(AVG(DATEDIFF(s.date, s.PreviousServiceDate)),0) AS
AvgDaysBetweenServices

FROM

    ta_maintenance AS m

LEFT JOIN

    ServiceIntervals AS s

    ON m.vehicle_id = s.vehicle_id

    AND m.date = s.date

GROUP BY

    m.vehicle_id

ORDER BY

    TotalMaintenanceCost DESC;

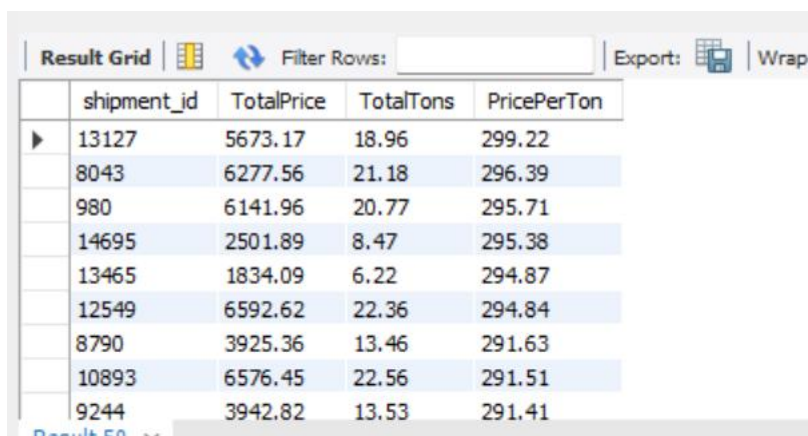
```

Result Grid			
Filter Rows:		Export:	Wrap Cell Con
	vehide_id	TotalMaintenanceCost	AvgDaysBetweenServices
▶	11	7328.94	139
	2	6744.65	127
	1	6633.15	64
	35	6492.09	115
	25	5887.97	84
	39	5432.04	132
	6	5399.02	76
	37	5229.18	168
	20	5193.84	153

Result 49 ×

-- 46. Pronađi sve pošiljke gde je odnos cene po toni veći od 100 EUR/t.

```
SELECT
    s.shipment_id,
    SUM(s.price_eur) AS TotalPrice,
    SUM(s.weight_tons) AS TotalTons,
    ROUND(SUM(s.price_eur) / SUM(s.weight_tons),2) AS PricePerTon
FROM
    ta_shipments AS s
GROUP BY
    s.shipment_id
HAVING
    SUM(s.weight_tons) > 0 AND
    SUM(s.price_eur) / SUM(s.weight_tons) > 100
ORDER BY
    SUM(s.price_eur) / SUM(s.weight_tons) DESC;
```



The screenshot shows a database query result grid with the following columns: shipment_id, TotalPrice, TotalTons, and PricePerTon. The results are ordered by PricePerTon in descending order. The first row is highlighted with a blue background.

shipment_id	TotalPrice	TotalTons	PricePerTon
13127	5673.17	18.96	299.22
8043	6277.56	21.18	296.39
980	6141.96	20.77	295.71
14695	2501.89	8.47	295.38
13465	1834.09	6.22	294.87
12549	6592.62	22.36	294.84
8790	3925.36	13.46	291.63
10893	6576.45	22.56	291.51
9244	3942.82	13.53	291.41

-- 47.. Izračunaj profit po ruti (shipment.price_eur - SUM(expenses.amount_eur)).

```
SELECT
```

```

r.route_id,

CONCAT(s.origin, ' - ', s.destination) AS RouteDescription,

SUM(s.price_eur) AS TotalRevenue,

SUM(e.amount_eur) AS TotalExpenses,

(SUM(s.price_eur) - SUM(e.amount_eur)) AS TotalProfit

FROM

    ta_routes AS r

LEFT JOIN

    ta_expenses AS e ON r.route_id = e.route_id

LEFT JOIN

    ta_shipments AS s ON r.shipment_id = s.shipment_id

GROUP BY

    r.route_id,

    s.origin,

    s.destination

ORDER BY

    TotalProfit DESC;

```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
route_id	RouteDescription	TotalRevenue	TotalExpenses	TotalProfit	
4948	Cummingsmouth - West Kimberly	7023.84	403.86	6619.98	
7419	West Tina - South Valerie	6510.27	0.00	6510.27	
13197	East Gary - North Toddborough	6507.41	0.00	6507.41	
5326	Larryville - Port Jillside	6494.09	0.00	6494.09	
12202	South Alicia - New Amandamouth	6454.18	0.00	6454.18	
8229	Katelynberg - East Natalie	6422.89	0.00	6422.89	
1436	Heiditown - Jacobtown	6327.24	0.00	6327.24	
3514	West Allisonville - New Jennifer	6326.63	0.00	6326.63	
8043	Jeffrevfort - Ashlevborough	6277.56	0.00	6277.56	

Result 51 x

-- 48. Kumulativan profit u svakoj godini, rast profita u odnosu na prethodni mesec, rast profita u odnosu na proslu godinu

WITH RevenuePerMonth AS (

SELECT

YEAR(s.date_requested) AS Year_,

MONTH(s.date_requested) AS Month_,

SUM(s.price_eur) AS TotalRevenue

FROM

ta_shipments AS s

INNER JOIN

ta_routes AS r ON s.shipment_id = r.shipment_id

WHERE

r.status = 'completed'

GROUP BY

YEAR(s.date_requested),

MONTH(s.date_requested)

),

ExpensesPerMonth AS (

SELECT

YEAR(e.date) AS Year_,

MONTH(e.date) AS Month_,

SUM(e.amount_eur) AS TotalExpenses

FROM

ta_expenses AS e

GROUP BY

YEAR(e.date),

MONTH(e.date)

),

-- ♦ Simulacija FULL JOIN pomoću UNION

Combined AS (

-- Deo 1: svi meseci sa prihodima

SELECT

r.Year_,

r.Month_,

r.TotalRevenue,

e.TotalExpenses

FROM

RevenuePerMonth AS r

LEFT JOIN

ExpensesPerMonth AS e

ON r.Year_ = e.Year_ AND r.Month_ = e.Month_

UNION

-- Deo 2: svi meseci sa troškovima

SELECT

e.Year_,

e.Month_,

r.TotalRevenue,

e.TotalExpenses

FROM

ExpensesPerMonth AS e

LEFT JOIN

RevenuePerMonth AS r

```
        ON r.Year_ = e.Year_ AND r.Month_ = e.Month_  
)
```

```
-- ♦ Glavni SELECT – isti kao originalni upit
```

```
SELECT
```

```
    Year_,
```

```
    Month_,
```

```
    TotalExpenses,
```

```
    TotalRevenue,
```

```
    (TotalRevenue - TotalExpenses) AS TotalProfit,
```

```
    (LAG(TotalRevenue - TotalExpenses) OVER (ORDER BY Year_, Month_)) AS  
    LastMonthProfit,
```

```
    (LAG(TotalRevenue - TotalExpenses) OVER (PARTITION BY Month_ ORDER BY Year_))  
    AS LastYearProfit,
```

```
    SUM(TotalRevenue - TotalExpenses) OVER (PARTITION BY Year_ ORDER BY Month_) AS  
    CumulativeProfit,
```

```
    (TotalRevenue - TotalExpenses)
```

```
    - LAG(TotalRevenue - TotalExpenses) OVER (ORDER BY Year_, Month_) AS  
    ProfitDifferenceFromPreviousMonth,
```

```
    (TotalRevenue - TotalExpenses)
```

```
    - LAG(TotalRevenue - TotalExpenses) OVER (PARTITION BY Month_ ORDER BY Year_)  
    AS ProfitDiffFromSameMonthLastYear
```

```
FROM
```

```
    Combined
```

```
ORDER BY
```

```
    Year_,
```

```
    Month_;
```

Result Grid									
Filter Rows:									
Export:									
Wrap Cell Content:									
Year_	Month_	TotalExpenses	TotalRevenue	TotalProfit	LastMonthProfit	LastYearProfit	CumulativeProfit	ProfitDifferenceFromPrev	
2023	1	151805.31	275715.21	123909.90	NULL	NULL	123909.90	NULL	
2023	2	134120.65	284834.24	150713.59	123909.90	NULL	274623.49	26803.69	
2023	3	180055.77	254942.75	74886.98	150713.59	NULL	349510.47	-75826.61	
2023	4	165028.53	302508.06	137479.53	74886.98	NULL	486990.00	62592.55	
2023	5	168288.45	312220.54	143932.09	137479.53	NULL	630922.09	6452.56	
2023	6	146041.57	365427.40	219385.83	143932.09	NULL	850307.92	75453.74	
2023	7	187690.80	274311.13	86620.33	219385.83	NULL	936928.25	-132765.50	
2023	8	154096.78	334716.15	180619.37	86620.33	NULL	1117547.62	93999.04	

-- 49. Prikazi sve troskove po godinama i mesecima po posebnim kolonama, a svaki trosak odrzavanja kao posebnu kolonu

SELECT

YEAR(e.date) AS Year_,

MONTH(e.date) AS Month_,

SUM(CASE WHEN e.type = 'Maintenance' THEN e.amount_eur ELSE 0 END) AS
MaintenanceCosts,

SUM(CASE WHEN e.type = 'Toll' THEN e.amount_eur ELSE 0 END) AS TollCosts,

SUM(CASE WHEN e.type = 'Other' THEN e.amount_eur ELSE 0 END) AS OtherCosts,

SUM(CASE WHEN e.type = 'Fuel' THEN e.amount_eur ELSE 0 END) AS FuelCosts,

SUM(CASE WHEN e.type = 'Fine' THEN e.amount_eur ELSE 0 END) AS FineCosts,

SUM(e.amount_eur) AS TotalCosts

FROM

ta_expenses AS e

GROUP BY

YEAR(e.date),

MONTH(e.date)

ORDER BY

YEAR(e.date),

MONTH(e.date);

Result Grid		Filter Rows:		Export:		Wrap Cell Content:		
	Year_	Month_	MaintenanceCosts	TollCosts	OtherCosts	FuelCosts	FineCosts	TotalCosts
▶	2023	1	29543.41	32060.84	30473.55	33037.74	26689.77	151805.31
	2023	2	30424.66	24287.71	27498.02	23576.88	28333.38	134120.65
	2023	3	37689.00	36115.55	35741.20	36196.45	34313.57	180055.77
	2023	4	36266.36	33681.98	29441.94	38210.66	27427.59	165028.53
	2023	5	33161.42	30285.32	35833.74	32849.60	36158.37	168288.45
	2023	6	25215.96	24567.07	32504.36	32093.44	31660.74	146041.57
	2023	7	43798.50	36053.12	32118.14	37679.50	38041.54	187690.80
	2023	8	31287.05	29922.19	22496.18	35660.39	34730.97	154096.78
	2023	9	34096.34	35308.66	36185.83	30806.93	30885.40	167283.16

Result 53 x

```
-- =====
-- CREATING VIEWS FOR POWER BI MODEL
-- =====
```

```
-- CumulativeProfitView
```

```
CREATE VIEW CumulativeProfitView AS (
WITH RevenuePerMonth AS (
    SELECT
        YEAR(s.date_requested) AS Year_,
        MONTH(s.date_requested) AS Month_,
        SUM(s.price_eur) AS TotalRevenue
    FROM
        ta_shipments AS s
        INNER JOIN
            ta_routes AS r ON s.shipment_id = r.shipment_id
    WHERE
        r.status = 'completed'
    GROUP BY
```

```

        YEAR(s.date_requested),
        MONTH(s.date_requested)
    ),
    ExpensesPerMonth AS (
        SELECT
            YEAR(e.date) AS Year_,
            MONTH(e.date) AS Month_,
            SUM(e.amount_eur) AS TotalExpenses
        FROM
            ta_expenses AS e
        GROUP BY
            YEAR(e.date),
            MONTH(e.date)
    ),

```

-- ♦ Simulacija FULL JOIN pomoću UNION

```

Combined AS (
    -- Deo 1: svi meseci sa prihodima
    SELECT
        r.Year_,
        r.Month_,
        r.TotalRevenue,
        e.TotalExpenses
    FROM
        RevenuePerMonth AS r
    LEFT JOIN
        ExpensesPerMonth AS e
        ON r.Year_ = e.Year_ AND r.Month_ = e.Month_

```

UNION

-- Deo 2: svi meseci sa troškovima

SELECT

e.Year_,

e.Month_,

r.TotalRevenue,

e.TotalExpenses

FROM

ExpensesPerMonth AS e

LEFT JOIN

RevenuePerMonth AS r

ON r.Year_ = e.Year_ AND r.Month_ = e.Month_

)

-- ♦ Glavni SELECT – isti kao originalni upit

SELECT

Year_,

Month_,

TotalExpenses,

TotalRevenue,

(TotalRevenue - TotalExpenses) AS TotalProfit,

(LAG(TotalRevenue - TotalExpenses) OVER (ORDER BY Year_, Month_)) AS
LastMonthProfit,

(LAG(TotalRevenue - TotalExpenses) OVER (PARTITION BY Month_ ORDER BY Year_))
AS LastYearProfit,

SUM(TotalRevenue - TotalExpenses) OVER (PARTITION BY Year_ ORDER BY Month_) AS
CumulativeProfit,

```

(TotalRevenue - TotalExpenses)
    - LAG(TotalRevenue - TotalExpenses) OVER (ORDER BY Year_, Month_) AS
ProfitDifferenceFromPreviousMonth,
(TotalRevenue - TotalExpenses)
    - LAG(TotalRevenue - TotalExpenses) OVER (PARTITION BY Month_ ORDER BY Year_)
AS ProfitDiffFromSameMonthLastYear
FROM
    Combined
ORDER BY
    Year_,
    Month_
);

```

-- CostsView

```

CREATE VIEW costsvie AS (
SELECT
    YEAR(e.date) AS Year_,
    MONTH(e.date) AS Month_,
    SUM(CASE WHEN e.type = 'Maintenance' THEN e.amount_eur ELSE 0 END) AS
MaintenanceCosts,
    SUM(CASE WHEN e.type = 'Toll' THEN e.amount_eur ELSE 0 END) AS TollCosts,
    SUM(CASE WHEN e.type = 'Other' THEN e.amount_eur ELSE 0 END) AS OtherCosts,
    SUM(CASE WHEN e.type = 'Fuel' THEN e.amount_eur ELSE 0 END) AS FuelCosts,
    SUM(CASE WHEN e.type = 'Fine' THEN e.amount_eur ELSE 0 END) AS FineCosts,
    SUM(e.amount_eur) AS TotalCosts
FROM
    ta_expenses AS e
GROUP BY

```

```

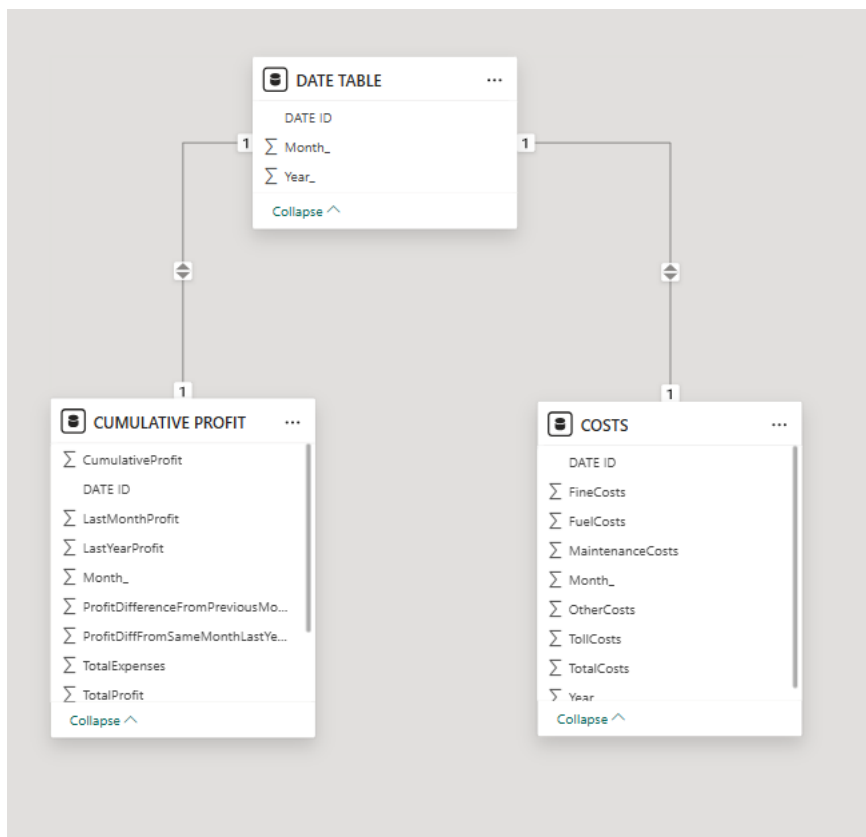
YEAR(e.date),
MONTH(e.date)
ORDER BY
    YEAR(e.date),
    MONTH(e.date)
);

```

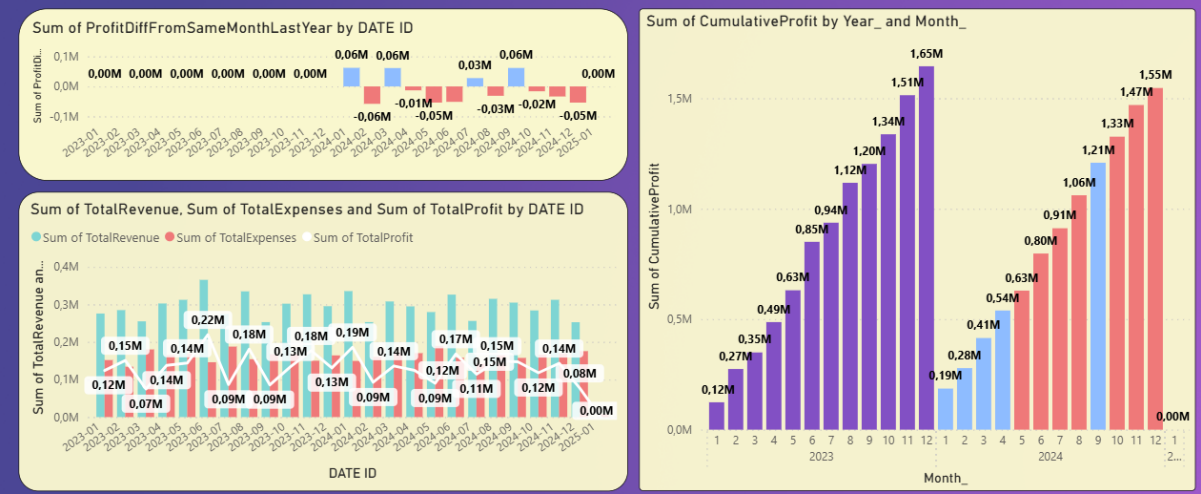
```

-- =====
-- PROFIT AND COSTS DASHBOARD PHOTOS
-- =====

```



Sum of TotalProfit	Sum of TotalRevenue	Sum of TotalExpenses
3,19M	7,10M	3,92M



Sum of TotalCosts	Sum of MaintenanceCosts	Sum of FuelCosts	Sum of TollCosts	Sum of OtherCosts	Sum of FineCosts
3,92M	803,38K	798,48K	762,11K	776,82K	777,93K

